Predictive and Generative AI for Loan Decisioning

This demo illustrates how a hybrid AI approach can be applied to loan approval workflows on IBM LinuxONE.

It includes the following:

- Predictive AI (XGBoost GBTree)
- Document processing and feature extraction using generative AI
- Retrieval-augmented generation (RAG)
- Generative AI recommendation and answers

Architecture Overview

At a high level, this demo does the following:

- Extract document features from an applicant's documents with the help of a generative AI model
- Use predictive AI to make a high/low risk prediction based on structured data.
- Produce a recommendation for approving or rejecting a loan application using the document features and prediction.
- Allow users to ask about the recommendation.

1. MULTI-MODAL DATA INGESTION
Structured Loan Application
Source: sample_loan.csv
• FICO Credit Score (e.g. 720)
• Debt-to-Income (DTI) Ratio
• Gross Annual Income
• Requested Loan Amount
• Credit Utilization & Purpose
Unstructured Financial PDFs
Applicant Verification Files
Bank Statements
Deposits, Balances & Debits
Pay Stubs
YTD Gross, Net & Deductions
W2 Tax Forms
Historical Annual Wages & Tax
Lending & Compliance Policy
Policy PDF Repository
• CFPB Truth in Lending (TILA)
• CFPB HMDA Guidance
• TRID Rule Requirements
• Adverse Action / ECOA / FCRA
• URLA Underwriting Guidelines
Enterprise Regulatory Base
2A. PREDICTIVE RISK ENGINE
XGBoost GBTree (PMML)
Module: loan_demo.model_utils
Runtime: PyPMML Engine
• High / Low Risk Binary Score
• Probability Calibration (P₀, P₁)
Ultra-low latency CPU inference
2B. DOC PARSING & COMPRESSION
PDF Normalizer & Evidence Extractor
Modules: pdf_utils.py + rag_utils.py
1. pypdf Text & Layout Extraction
2. Regex / NFKD ASCII Normalization
3. Financial Evidence Synthesizer
Cashflow, YTD Net/Gross, Tax Ratio
2C. BM25 POLICY INDEX & RAG
Policy Chunking & Ranking
Engine: rank_bm25.BM25Okapi
• Policy Concatenation & Chunking
• Dynamic Query Formulation
Top-K Context Retrieval
Injects authoritative regulatory rules
3. GEN AI FEATURE EXTRACTION
IBM Granite Reasoning
Model: Granite-3.3-2B-Instruct
Framework: llama-cpp-python
Prompt Reasoning Task:
• Financial Stability Evaluation
• Cash Flow Trends (Pos/Neut/Neg)
• Income Consistency & Compliance
Zero-shot structured assessment
Structured Financial Features
Granite Output Schema:
Financial Stability
High / Medium / Low
Closing vs Opening balance delta
Cash Flow Stability
Positive / Neutral / Negative
Credits vs Debits monthly velocity
Income Consistency
High / Medium / Low
Paystub YTD vs W2 annual alignment
Compliance Status
Compliant / Review Needed
Standard mortgage doc validity
Enriched Underwriting Vector
4. HYBRID AI DECISION & EXPLAINABILITY
Multi-Modal Recommendation Synthesizer
Context Fusion: Applicant + Risk Score + Doc Analysis + Policies
Applicant Profile (FICO, DTI, Income, Utilization, Age)
XGBoost Risk Prediction (Low/High Risk + Probabilities)
Granite Extracted Features (Stability, Cash Flow, Consistency)
BM25 Top Relevant Policy Clauses & Underwriting Criteria
Recommendation: Approve / Reject / Review
Conversational Explainability (Q&A)
Underwriter / Auditor Interactive Inquiry Interface
Underwriter Query:
"Why was this applicant considered low risk?"
Granite Grounded Explanation:
"The applicant demonstrates a high FICO score (720), low DTI ratio, and positive monthly cashflow. Consistent W2 wages and compliant verification documents align directly with the TRID & CFPB guidelines for automated prime approval."
✓ Auditable, compliant, transparent reasoning
Zero Data Egress / Fully On-Premises
All inference executed locally on IBM LinuxONE without public cloud calls

Environment Setup

Refer to the code in `loan_demo` to see the specifics.

[2]:
```python
import pandas as pd

from loan_demo.model_utils import predict_risk
from loan_demo.pdf_utils import extract_text_from_pdf
from loan_demo.rag_utils import (
    load_model,
    load_policy_docs,
    compress_applicant_documents,
    extract_document_features,
    generate_recommendation,
    generate_answer,
    build_policy_index,
    summarize_applicant,
)
```

[3]:
```python
# Load IBM Granite model used for generative AI
model = load_model()
```

```
llama_kv_cache: the V embeddings have different sizes across layers and FA is not enabled - padding V cache to 256
```

IBM zDNN / NNPA Hardware Acceleration — llama.cpp

The IBM Granite model is served via **llama.cpp**, compiled with two hardware acceleration backends:

Backend	Flag	What it accelerates
zDNN / NNPA	`-DGGML_ZDNN=ON`	Matrix multiplications in transformer attention and FFN layers, dispatched to the on-chip Neural Network Processing Assist facility on z16
OpenBLAS	`-DGGML_BLAS=ON`	Large GEMM operations using VXE/VXE2 SIMD vector extensions

Both backends are **transparent** — llama.cpp selects the accelerated path automatically at runtime based on tensor shape and hardware availability. The cell below verifies the build flags and confirms the zDNN shared library is present.

[4]:
```python
import glob, os, importlib.metadata

# ── 1. llama_cpp_python version ──────────────────────────────────────────
llama_version = importlib.metadata.version('llama_cpp_python')
print(f'llama_cpp_python version : {llama_version}')

# ── 2. Check zDNN shared library is present in llama_cpp lib dir ─────────
zdnn_libs = glob.glob(
    '/opt/venv/lib*/python3.12/site-packages/llama_cpp/lib/libggml-zdnn*.so'
)
if zdnn_libs:
    for lib in zdnn_libs:
        size_mb = os.path.getsize(lib) / 1024**2
        print(f'zDNN shared library      : {lib}  ({size_mb:.1f} MB)')
else:
    print('WARNING: libggml-zdnn.so not found — NNPA acceleration inactive')

# ── 3. Confirm build metadata from llama_cpp ─────────────────────────────
try:
    from llama_cpp import llama_cpp as _lc
    supports_gpu = getattr(_lc, 'llama_supports_gpu_offload', None)
    print(f'GPU offload support      : {supports_gpu() if supports_gpu else "n/a"}')
except Exception:
    pass

# ── 4. Check the model file is the correct s390x build ──────────
from loan_demo.rag_utils import MODEL_PATH
if os.path.exists(MODEL_PATH):
    size_mb = os.path.getsize(MODEL_PATH) / 1024**2
    # Read GGUF magic and verify it is a valid GGUF file
    with open(MODEL_PATH, 'rb') as f:
        magic = f.read(4)
    valid = magic == b'GGUF'
    print(f'Granite model            : {MODEL_PATH}')
    print(f'  Size                   : {size_mb:.0f} MB')
    print(f'  GGUF magic             : {magic}  ({"valid" if valid else "INVALID — wrong file?"})')
    print(f'  Expected               : granite-3.3-2b-instruct-be.Q4_K_M (s390x)')
else:
    print(f'WARNING: model file not found at {MODEL_PATH}')

print()
print('Build flags used for this container:')
for flag in [
    '-DGGML_ZDNN=ON        (IBM zDNN / NNPA hardware acceleration)',
    '-DGGML_BLAS=ON        (OpenBLAS SIMD via VXE/VXE2)',
    '-DGGML_BLAS_VENDOR=OpenBLAS',
    '-DCMAKE_C_FLAGS=-march=z16',
    '-DCMAKE_CXX_FLAGS=-march=z16',
]:
    print(f'  {flag}')
```

```
llama_cpp_python version : 0.3.36
zDNN shared library      : /opt/venv/lib/python3.12/site-packages/llama_cpp/lib/libggml-zdnn.so  (0.0 MB)
zDNN shared library      : /opt/venv/lib64/python3.12/site-packages/llama_cpp/lib/libggml-zdnn.so  (0.0 MB)
GPU offload support      : False
Granite model            : /home/ibm-user/granite-4.0-h-350m-base-be.f16.gguf
  Size                   : 653 MB
  GGUF magic             : b'GGUF'  (valid)
  Expected               : granite-3.3-2b-instruct-be.Q4_K_M (s390x)

Build flags used for this container:
  -DGGML_ZDNN=ON        (IBM zDNN / NNPA hardware acceleration)
  -DGGML_BLAS=ON        (OpenBLAS SIMD via VXE/VXE2)
  -DGGML_BLAS_VENDOR=OpenBLAS
  -DCMAKE_C_FLAGS=-march=z16
  -DCMAKE_CXX_FLAGS=-march=z16
```

Load Applicant Documents

Select applicant files and prepare content for feature extraction. Refer to `sample_data` to choose an applicant to process.

[5]:
```python
APPLICANT_ID = 5

loan_df = pd.read_csv("sample_data/sample_loan.csv")
loan = loan_df.iloc[APPLICANT_ID - 1].to_dict()

bank_pdf = (
    f"sample_data/bank_statements/"
    f"{APPLICANT_ID}.pdf"
)

paystub_pdf = (
    f"sample_data/pay_stubs/"
    f"{APPLICANT_ID}.pdf"
)

w2_pdf = (
    f"sample_data/w2/"
    f"{APPLICANT_ID}.pdf"
)

bank_text = extract_text_from_pdf(bank_pdf)
paystub_text = extract_text_from_pdf(paystub_pdf)
w2_text = extract_text_from_pdf(w2_pdf)

# additional preprocessing and cleanup of extracted text
document_text = compress_applicant_documents(
    bank_text,
    paystub_text,
    w2_text,
)

print(document_text)
```

```
Bank: opening=25,588.90, closing=146,746.07, credits=167,624.44, debits=46,467.27, cashflow=positive

Paystub: ytd_gross=264,633.60, ytd_deductions=61,262.68,ytd_net=203,370.92, gross_pay=24,259.72, net_pay=18,643.59

W2 Latest: wages=282,112.20, federal_tax=62,064.68
```

Extract Financial Features

The LLM analyzes applicant information and produces a set of financial features.

[6]:
```python
# Use the model to extract features from unstructured documents
document_features = (
    extract_document_features(
        model,
        document_text,
    )
)
```

```
Financial Stability: High
Cash Flow Stability: High
Income Consistency: High
Compliance Status: Compliant

Evidence:
Bank: opening=25,588.90, closing=146,746.07, credits=167,624.
```

Predict Risk

An XGBoost GBTree model generates a traditional risk score using applicant features

[7]:
```python
risk_result = predict_risk(loan)
print(risk_result)
```

```
{'risk': 'Low Risk', 'probability_low': 0.9973529772689301, 'probability_high': 0.0026470227310690002}
```

Retrieve Relevant Lending Policies

Using BM25 retrieval, we identify supporting policy guidance for the recommendation step. This strategy allows a model like IBM Granite to have access to a huge volume of information while only retrieving the information most relevant.

[8]:
```python
policies = load_policy_docs("sample_data/policy")

bm25, policy_chunks = build_policy_index(
    policies,
    chunk_size=300,
)
```

Recommendation

Now we combine the analysis of the applicant's financial documents, the risk model prediction, and the retrieved policy guidance to produce a final recommendation in natural language.

[9]:
```python
recommendation = generate_recommendation(
    model=model,
    applicant=loan,
    risk_result=risk_result,
    document_features=document_features,
    bm25=bm25,
    policy_chunks=policy_chunks,
)
```

```
Recommendation: Approve
Main Reason: High credit score, low DTI and stable documented income meet policy.
```

Ask Questions About the Application

Natural-language question answering over the generated decision and context.

[10]:
```python
# Preprocessing step as many fields are not relevant
applicant_summary = summarize_applicant(loan)

# Build context for model
context = f"""
Applicant:
{applicant_summary}

Risk:
{risk_result}

Document Analysis:
{document_features}

Recommendation:
{recommendation}
"""

# Change this question to ask different questions
response = generate_answer(
    model,
    context,
    "Why was this applicant considered low risk?"
)
```

```
 The applicant has a high credit score and strong income relative to the loan amount. Documents show positive cash flow, consistent income and compliance,
and the risk model predicts low risk.
```